

When More Replicas Do Not Help:

Testing File-Storage Replication Intuition in Distributed SQL

Anonymous Author(s)

Affiliation omitted in working draft

Abstract

Replication-factor adaptation is well motivated in distributed file storage, where an additional block copy may improve locality, distribute concurrent reads, or protect another failure domain. This paper investigates whether the same tuning intuition transfers to a consensus-based distributed SQL database. CockroachDB is the primary object of study; HDFS is used as an architectural and experimental contrast that exposes why replication can act as a read-scaling mechanism in file storage. In CockroachDB, replicas participate in Raft groups, transactions, and leaseholder-directed execution, so an additional voting replica does not automatically create an independent path for strongly consistent reads. Rather than proposing a new adaptation algorithm, we first test whether RF is an effective and predictable database control action. A validated local CockroachDB ladder covering RF 3, 5, and 7 completed all 27 runs without workload or collection errors. Read-only and most 80:20 runs reached a configured 150 QPS cap, while the saturated 50:50 series was variable and non-monotonic. A smaller HDFS contrast showed that reader parallelism, rather than RF, dominated local throughput. These preliminary results do not justify generic database RF adaptation. They instead identify the conditions under which it may be useful: failure-domain requirements, deliberate leaseholder placement, eligible follower reads, geographic locality, and a workload stable enough to amortize reconfiguration.

Keywords: replication factor, distributed SQL, CockroachDB, HDFS, Raft, distributed storage, performance evaluation

1 Introduction

Replication level and replica placement are established control mechanisms in distributed file systems. The Google File System (GFS) stores chunks across commodity machines to provide fault tolerance and high aggregate performance [4]. HDFS similarly exposes a configurable per-file replication factor and combines rack-aware placement with nearest-replica selection [7]. Later systems made replication explicitly workload-aware. Scarlett creates additional replicas for popular content in MapReduce clusters [1], while Copyset Replication shows that the placement pattern of a fixed number of replicas can materially change durability under

correlated failures [2]. This work creates a plausible systems intuition: if demand becomes read-heavy or geographically skewed, create more replicas near that demand.

This paper asks whether that intuition transfers to a distributed SQL database. In a file system, another block replica can become another physical read source. In a database such as CockroachDB, another voting replica joins a consistency and transaction protocol. Raft orders replicated state-machine updates through a leader and commits log entries after majority agreement [6]. CockroachDB applies this model to key ranges and combines it with leases and distributed transaction processing [8]. Increasing RF therefore changes failure tolerance, replication fan-out, majority size, and placement choices, but it does not automatically create another independent path for strongly consistent reads.

CockroachDB is the research target, while HDFS provides the contrast. The HDFS experiment is not a product benchmark. It illustrates the mechanism from which RF tuning intuition originates and provides a reference for interpreting database behavior. We intentionally stop before algorithm design: an adaptive controller is useful only if changing RF produces a repeatable benefit, the workload remains in that regime long enough to amortize reconfiguration, and the controller can observe the mechanism causing the benefit.

This paper makes three contributions:

1. It evaluates RF as a performance-control action in a consensus-based distributed SQL database.
2. It uses file storage to explain why another replica can improve reads in one architecture but does not automatically do so in a database.
3. It provides a reproducible experimental scaffold and a mechanism-based decision framework for future database RF adaptation.

The evidence is preliminary and local. The contribution is therefore not a claim that one RF is universally optimal. It is a narrower result: the present measurements do not show a stable monotonic performance benefit from increasing CockroachDB RF, and the architecture explains why such a benefit requires additional conditions.

2 Background and Prior Work

2.1 Replication in file storage

HDFS separates metadata coordination from the data path. The NameNode manages the namespace, block locations, replication decisions, and placement policy. DataNodes store blocks and serve client reads and writes. After obtaining block locations, clients transfer data directly to or from a DataNode [7]. Another replica can therefore become a closer read source, distribute concurrent demand, or cover an additional failure domain.

These benefits remain conditional. They depend on physical placement, topology, cache state, scheduling, and sufficient concurrent demand. Higher RF also consumes storage, sends more replication traffic, expands the write pipeline, and creates background work when the setting changes. Scarlett demonstrates the positive case: long-lived content popularity can justify selective block replication because the new copies relieve real storage-node hotspots [1].

2.2 Consensus-based distributed SQL

Raft manages a replicated log and tolerates failures by committing entries after a majority of voters acknowledges them [6]. For n voters, a majority contains $\lfloor n/2 \rfloor + 1$ replicas. Increasing RF changes both the replication fan-out and, at some transitions, the majority size. This is not the passive-copy model implied by file storage.

CockroachDB partitions data into ranges, each represented by a Raft replication group [8]. A Raft leader orders log entries, while a range leaseholder coordinates most strong reads and proposes writes. The leader and leaseholder are related operational roles but should not be conflated conceptually. In particular, a strong read does not consult a quorum for every request, yet arbitrary voting replicas do not become generic strong read servers: requests are normally routed through the leaseholder. Follower reads can use non-leaseholder replicas, including geographically close ones, but they introduce a different freshness contract.

Geo-distributed databases also demonstrate that replica count cannot be separated from placement and consistency. Spanner combines synchronous replication with explicit geographic configuration and globally meaningful time [3]. Its design illustrates both sides of the trade-off: replicas can place data near users and survive regional failures, while a geographically distributed write quorum can put wide-area delay on the commit path. More generally, classical work warns that replication can create substantial coordination cost when updates must remain consistent [5].

2.3 Positioning

Prior file-storage work asks how to place or add copies once replicas are known to be useful read-serving objects. Prior database work explains consensus, trans-

actions, and geo-distribution. This paper connects the two by asking a step that precedes controller design: is changing the number of database replicas itself a useful and predictable performance action? The HDFS experiment is included only to expose the architectural source of the file-storage intuition.

3 Research Questions

The study is organized around three research questions:

- RQ1.** Does increasing RF improve CockroachDB throughput or latency under strong read-only and mixed read/write workloads?
- RQ2.** Do the database results exhibit the read-scaling behavior that motivates RF adaptation in file storage?
- RQ3.** Under which consistency, placement, workload-duration, and geographic conditions could RF adaptation still be worthwhile in distributed SQL?

Table 1 summarizes the mechanism-level contrast. The important distinction is not that file systems always benefit from higher RF. It is that another file block copy is directly eligible to serve reads, whereas another database voter is primarily part of a replicated state machine.

4 Experimental Methodology

4.1 Primary database experiment

The local CockroachDB deployment contains nine Docker containers on one Windows host. A synthetic benchmark table contains 90,000 rows and is configured as one range to make the tested replication group explicit. The table is recreated before each run. The workload generator addresses all nine nodes with two workers per node and uses strong reads.

The validated safe ladder varies:

- RF values 3, 5, and 7;
- read/write ratios 100:0, 80:20, and 50:50;
- three repetitions per configuration;
- 90 seconds of measured workload and 30 seconds of cooldown; and
- a total target rate of 150 QPS.

The 27 configurations are shuffled to reduce monotonic-order bias from thermal effects, background compaction, and replica movement. Each run records actual QPS, successful and failed operations, connection errors, and collection errors. The result directory is accepted only if the full RF-ratio-repetition matrix exists, all workload error counts are zero, load files are readable, metrics contain no error rows, and all nine containers remain running.

The ladder used a legacy native-metrics collector that stored already-computed cumulative p90 and p99 snapshots from one node. Such values combine activity from before the workload and are not cluster-wide per-run latency. They are preserved as diagnostic data but excluded from the paper. The corrected collector

Table 1: The same replication-factor parameter activates different mechanisms in file storage and consensus-based distributed SQL.

Tuning question	File-storage intuition	Distributed SQL implication
Can another replica serve reads?	Normally yes, subject to placement and client selection.	Not for arbitrary strong reads; leaseholder or follower-read rules apply.
What does RF primarily change?	Number and location of physical block copies.	Consensus membership, failure tolerance, placement, and replication fan-out.
Where can read performance improve?	Locality, parallel service, and storage-node load distribution.	Leaseholder placement or eligible local follower reads.
What is the main incremental cost?	Storage, write pipeline, transfer, and re-replication.	Storage, write replication, majority coordination, and range rebalancing.

records raw cumulative histogram buckets and counters from all nodes, then computes end-minus-start deltas over the workload interval. A repeat run is required before native SQL/KV latency can be reported.

The 150 QPS cap is also important. A run that reaches the target proves that the configuration sustains the selected load; it does not estimate maximum capacity. Conversely, low achieved QPS is reported as saturation rather than silently treating target throughput as controlled.

4.2 File-storage contrast

The HDFS deployment contains one NameNode, five DataNodes, and one client container on the same physical host. The seed sweep varies RF from 1 to 5, uses four synthetic 64 MiB files, and measures 1, 2, 4, and 8 parallel readers. Every combination is repeated twice in randomized order. For each RF, the benchmark writes the data set, waits for `hdfs dfs -setrep -w` to complete, and then measures aggregate read throughput.

The HDFS data are accepted only if the complete RF-reader-repetition matrix exists and all write and read measurements are positive. This experiment is deliberately smaller than the database ladder. Its role is to show the distinct read-serving mechanism, not to rank two different products.

4.3 Analysis and evidence classes

The repository distinguishes smoke tests, preliminary local evidence, and paper-grade evidence. This paper reports validated preliminary evidence. Means and sample standard deviations are computed across repetitions, but the small sample sizes do not justify formal significance tests. Raw result directories retain meta-data, command parameters, logs, and per-run outputs. Compact per-run and grouped CSV summaries are curated with the paper.

The failed local CockroachDB RF=9 attempt is excluded from performance curves. It destabilized the nine-container setup and is retained only as evidence of an environment limit.

5 Results

5.1 CockroachDB target attainment

All 27 planned runs completed with zero connection, read, write, or metrics errors. Table 2 reports actual QPS as mean and sample standard deviation across three repetitions.

Table 2: CockroachDB actual QPS, mean (standard deviation), $n = 3$.

Read:write	RF=3	RF=5	RF=7
100:0	149.71 (0.30)	149.71 (0.27)	149.47 (0.67)
80:20	149.37 (0.57)	149.80 (0.16)	145.24 (6.00)
50:50	133.20 (6.84)	138.50 (13.33)	129.86 (16.14)

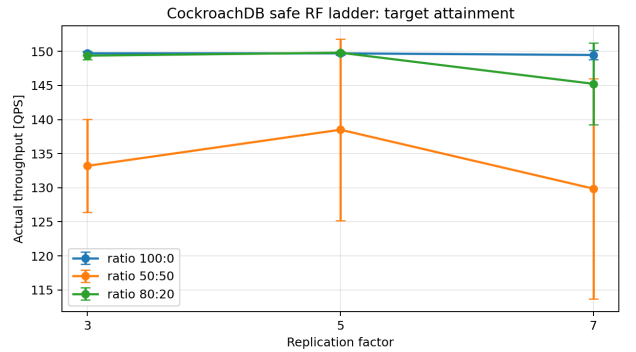


Figure 1: CockroachDB target attainment. Error bars show one sample standard deviation. The 150 QPS target caps read-only and most 80:20 observations.

The read-only workload reaches the 150 QPS target at every RF with very little variation. This demonstrates stability at the selected load, but cannot establish equal maximum capacity or a read-scaling benefit. RF=3 and RF=5 similarly reach the target in the 80:20 workload, while RF=7 averages 145.24 QPS and varies more strongly.

The 50:50 workload exposes a clearer saturation regime. Mean throughput is non-monotonic and variation is substantial. RF=5 has the highest mean, but its standard deviation is nearly twice that of RF=3. RF=7

has both the lowest mean and the largest spread. No increasing RF sequence provides a stable monotonic improvement.

The result is consistent with the architectural expectation: voting replicas do not act as independent strong-read servers and can interact with consensus work, leaseholder placement, range reconfiguration, and shared-host contention. It does not prove that RF=3 is optimal; the capped and small experiment cannot support that claim.

5.2 HDFS contrast

The HDFS seed completed all 40 planned measurements and passed validation. Table 3 reports mean read throughput across two repetitions.

Table 3: HDFS mean read throughput in MiB/s, $n = 2$.

RF	1 reader	2 readers	4 readers	8 readers
1	14.67	26.13	40.69	49.73
2	15.87	25.05	38.59	43.92
3	16.56	27.86	40.06	48.82
4	17.48	27.24	40.46	53.24
5	16.21	26.01	39.87	50.12

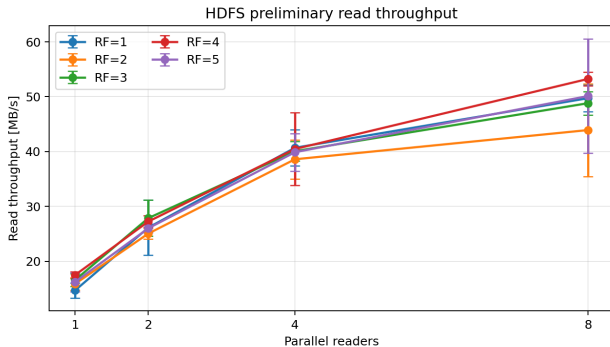


Figure 2: HDFS read throughput. Reader parallelism dominates the local result; RF remains smaller and non-monotonic. Error bars show one sample standard deviation.

The clearest pattern is scaling with reader count, not with RF. Moving from one to eight readers increases throughput by roughly three times across all RF settings. The RF effect is smaller and non-monotonic: RF=4 is best for one and eight readers, while RF=3 is best for two readers and RF=1 is close to the best result for four readers.

This does not establish that increasing RF always improves HDFS performance. It establishes the narrower architectural contrast: another block copy is eligible to serve a nearby or concurrent reader, even though the shared-host experiment does not isolate locality or storage-node load distribution. Strong CockroachDB reads lack a corresponding generic path.

5.3 Answers to the research questions

RQ1. The preliminary ladder provides no evidence of a stable monotonic CockroachDB throughput benefit as RF increases. Capped runs establish target attainment only, while saturated 50:50 runs are non-monotonic and variable. Valid latency results remain pending.

RQ2. The file-storage intuition does not transfer directly. File replicas are potential read sources; database voters are consensus participants. The current HDFS data do not show a monotonic RF benefit either, but they expose a different read-serving mechanism.

RQ3. Database RF adaptation is plausible only when it activates a concrete benefit: required failure-domain coverage, improved leaseholder placement, eligible follower reads, useful geographic locality, or a persistent workload that can amortize range replication and rebalancing.

6 Implications for RF Tuning

6.1 Resilience is the default benefit

The most direct reason to increase CockroachDB RF is a required failure-tolerance level. More voters can cover additional node, zone, or region failures. Replica count alone is insufficient, however: Copysets illustrates that the correlation structure of placement can matter as much as the nominal number of copies [2]. An operator should therefore state the decision as a failure-domain and placement requirement, not merely as a larger integer.

6.2 Eligible read paths

The file-storage contrast exposes the missing mechanism. In HDFS, the client can select a nearby block replica and concurrent readers can distribute work over DataNodes. CockroachDB obtains an analogous locality benefit only when reads are eligible to use the added replica. This can occur through deliberate leaseholder placement or follower reads with acceptable staleness. Without such a path, increasing RF for a read-heavy strong-consistency workload has no clear reason to increase read-serving capacity.

6.3 Geographic benefits and costs

Geography can make a database replica valuable when placement converts remote access into local access. It also makes coordination more expensive. A remote voter may improve regional survivability while putting wide-area latency on the write quorum. A nearby replica improves foreground reads only when the leaseholder is placed appropriately or the application accepts follower-read freshness. Strong leaseholder reads, bounded-staleness follower reads, and geographically distributed writes must therefore be evaluated separately.

6.4 Amortizing reconfiguration

Dynamic RF changes create replicas, move range data, and trigger rebalancing. A controller reacting to a short burst may finish only after the burst disappears. Reconfiguration can also overlap with lease transfers, range movement, compaction, and background maintenance. A useful policy must estimate both the magnitude and the duration of the expected benefit.

Conceptually, increasing database RF is defensible only when

$$B_{\text{resilience}} + B_{\text{locality}} > C_{\text{storage}} + C_{\text{write}} + C_{\text{migration}} + C_{\text{coordination}}. \quad (1)$$

The terms need not use one monetary unit in an implementation, but the inequality forces the policy to name the active benefit. “Read-heavy” alone is not enough.

6.5 Next evidence needed

A paper-grade database evaluation should:

1. use at least 180-second randomized runs with at least three, preferably five, repetitions;
2. sweep offered load to locate capacity boundaries instead of capping all read-heavy cases at one target;
3. report client p50/p90/p99 latency and cluster-wide native SQL/KV p90/p99 from per-run histogram deltas;
4. compare strong reads with a separately labeled follower-read series; and
5. use a multi-host or controlled-network setup for geographic claims.

7 Threats to Validity

Shared host. All logical nodes share one physical CPU, memory system, disk subsystem, and network stack. Docker Desktop and the host scheduler can produce contention that would differ on independent machines. The setup cannot demonstrate rack or regional locality.

Workload scope. The CockroachDB workload uses one synthetic table, one range, controlled operation mixes, and uniform target rates. Real applications may include multiple ranges, secondary indexes, multi-statement transactions, skew, and application-level think time.

Target-rate censoring. Read-only and most 80:20 runs reach the 150 QPS cap. They show successful target attainment, not maximum throughput. Only the lower achieved rates provide saturation evidence.

Latency instrumentation. Native latency snapshots from the safe ladder are cumulative and single-node. Excluding them avoids a false per-run claim, but leaves RQ1 only partially answered until the corrected collector is used.

Reconfiguration effects. Changing RF can cause replica movement, lease transfer, compaction, and

other background activity. Cooldown and randomization reduce but do not remove these effects.

File-storage contrast. The HDFS seed uses two repetitions, synthetic zero-filled files, and one shared host. It does not prove that higher RF generally improves file-system reads. It only demonstrates the architectural availability of multiple block sources.

External validity. Conclusions concern this CockroachDB version, configuration, strong-read mode, and local environment. They should not be generalized to all distributed SQL systems or all replica types.

8 Conclusion

This paper studies replication-factor tuning in distributed SQL by testing an intuition inherited from file storage. In a file system, another block replica may become another read source. In CockroachDB, another voting replica primarily changes fault tolerance, placement, and replicated state-machine topology. It improves read performance only when leaseholder placement or follower-read behavior makes the replica eligible to serve useful demand.

The validated preliminary ladder completed without errors but showed no stable monotonic throughput improvement as RF increased from 3 to 7. Read-heavy runs were capped by the offered load, and the saturated 50:50 series was variable and non-monotonic. These results do not justify a new generic database RF adaptation algorithm. They support a more careful rule: change RF only when a named benefit—failure-domain coverage, leaseholder placement, eligible local follower reads, or consistency-aware geographic placement—can plausibly exceed storage, write, coordination, and reconfiguration costs.

Artifact Availability

The experiment runners, validation scripts, curated preliminary CSV files, and figure-generation code are available at <https://github.com/janraj00/replication-factor-study>. Raw generated result directories remain separate from the repository; compact summaries and metadata document the evidence used here.

References

- [1] Ganesh Ananthanarayanan, Sameer Agarwal, Srikanth Kandula, Albert Greenberg, Ion Stoica, Duke Harlan, and Ed Harris. Scarlett: Coping with skewed content popularity in mapreduce clusters. In *Proceedings of the Sixth European Conference on Computer Systems*, EuroSys ’11, pages 287–300. ACM, 2011. doi: 10.1145/1966445.1966472.
- [2] Asaf Cidon, Stephen M. Rumble, Ryan Stutsman, Sachin Katti, John Ousterhout, and Mendel Rosenblum. Copysets: Reducing the frequency of data loss in cloud storage. In *2013 USENIX*

- Annual Technical Conference*, USENIX ATC '13, pages 37–48. USENIX Association, 2013. URL <https://www.usenix.org/conference/atc13/technical-sessions/presentation/cidon>.
- [3] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, J. J. Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander Lloyd, Sergey Melnik, David Mwaura, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Yasushi Saito, Michal Szymaniak, Christopher Taylor, Ruth Wang, and Dale Woodford. Spanner: Google’s globally-distributed database. In *10th USENIX Symposium on Operating Systems Design and Implementation*, OSDI '12, pages 251–264. USENIX Association, 2012. URL <https://www.usenix.org/conference/osdi12/technical-sessions/presentation/corbett>.
- [4] Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung. The google file system. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles*, SOSP '03, pages 29–43. ACM, 2003. doi: 10.1145/945445.945450.
- [5] Jim Gray, Pat Helland, Patrick O’Neil, and Dennis Shasha. The dangers of replication and a solution. In *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*, SIGMOD '96, pages 173–182. ACM, 1996. doi: 10.1145/233269.233330.
- [6] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *2014 USENIX Annual Technical Conference*, USENIX ATC '14, pages 305–319. USENIX Association, 2014. URL <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>.
- [7] Konstantin Shvachko, Hairong Kuang, Sanjay Raddia, and Robert Chansler. The hadoop distributed file system. In *2010 IEEE 26th Symposium on Mass Storage Systems and Technologies*, MSST '10, pages 1–10. IEEE, 2010. doi: 10.1109/MSST.2010.5496972.
- [8] Rebecca Taft, Irfan Sharif, Andrei Matei, Nathan VanBenschoten, Jordan Lewis, Tobias Grieger, Kai Niemi, Andy Woods, Anne Birzin, Raphael Poss, Paul Bardea, Amruta Ranade, Ben Darnell, Bram Gruneir, Justin Jaffray, Lucy Zhang, and Peter Mattis. Cockroachdb: The resilient geo-distributed sql database. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, SIGMOD '20, pages 1493–1509. ACM, 2020. doi: 10.1145/3318464.3386134.